

BIM and ReadSeal: Safe Early Reuse of Shared GPU Buffers Across Processes

Xingbo Hui, Xuetao Jiang, Guodong Ye, Ruochen Shao, Tianyi Wang, Sicheng Wu, Rui Zhou*, Qingguo Zhou*
School of Information Science and Engineering, Lanzhou University, Lanzhou, China

*Co-corresponding authors: zr@lzu.edu.cn, zhouqg@lzu.edu.cn

Abstract—Processes in a perception pipeline often share one large GPU tensor ~~instead of copying it for every recipient. When may the producer rather than giving each a private copy. The producer must then decide when it may overwrite the shared buffer with the next frame?~~. CUDA stream-only release ~~waits only for reuses the buffer once all submitted GPU work, so it can hand the next frame to a reader has finished, so a recipient that has not started can silently compute on the next frame. Full retention is safe but waits for every result and refuses, refusing new frames while the GPU sits idle. Our key insight is is idle. We observe that a shared tensor has two lifetimes: its storage is needed only until the last read completes, but whereas its frame identity must last until every result is delivered. BEV-in-Memory (BIM) registers every recipient process when a frame is published, protecting recipients that have not started, so no recipient loses its frame before it starts reading. Inside each recipient, ReadSeal analyzes storage sharing in each the model graph to find where the model stops reading the last operation that reads the shared tensor, and refuses to run a plan that no longer matches execution the executing code, weights, or inputs. Recipients then finish on private intermediate results while the buffer already holds the next frame. In an A100 pipeline with four early-reading recipients, one BIM slot buffer delivers 1.31–1.50× the correct, timely results of one-slot full retention—full retention with one buffer at 25–40 Hz and, at 30 Hz, matches two full-retention slots with a tuned request cap, at as many as full retention with two buffers and a tuned cap on outstanding requests, with 8–11 ms more mean and 15–19 ms more 95th-percentile latency. Derived read boundaries deliver as much as hand-placed ones and are rebuilt automatically after when the model changes. Ordered tests show that stream-only release serves a pending reader. In ordered tests, a reader that submitted its work late computed on the next frame, while BIM does not under stream-only release, but never under BIM.~~

Index Terms—GPU buffer reuse, zero-copy IPC, buffer lifetime, alias analysis, autonomous driving.

I. INTRODUCTION

Consider a driving stack built on Autoware or another framework based on ROS 2, the Robot Operating System [1], [2]. A camera–LiDAR fusion node publishes a bird’s-eye-view (BEV) feature map, a top-down grid of features around the vehicle. ~~Separate, and separate~~ processes for 3D detection, tracking, motion planning, and data recording read it on their own schedules (Fig. 1a). We call ~~these processes each such process a recipients recipient and the components within them that access the map each component inside a recipient that reads the map a readers reader. For example, BEVFusion; BEVFusion, for example,~~ feeds one BEV map to both a detection and a map-segmentation head [3]. Our evaluated

map is a $1 \times 512 \times 180 \times 180$ fp32 tensor of 63.3 MiB [3]. Private copies for four recipients ~~would~~ add 256 MiB of device memory (64 MiB each after allocator rounding), ~~a cost that grows and at 30 Hz the copying alone would read and write about 15 GiB/s. These costs grow~~ with the number of recipients and resident frames. ~~The cost matters frames in flight, and they matter~~ most on automotive systems-on-chip, whose GPU shares DRAM with the rest of the stack [4]. Zero-copy ~~middleware lets inter-process communication (IPC) avoids them by letting recipients map the sender producer’s buffer [5], [6], and recent ROS 2 facilities extend this to GPU memory [7], [8]. The producer must then decide when it may overwrite that buffer, which we call a slot, with the next frame.~~

Two common release rules ~~leave a gap err in opposite directions~~ (Section II). *CUDA stream-only release*, ~~shortened to stream-only in the figures for short,~~ reuses the slot once all GPU work submitted so far has finished. A ~~queued recipient that has not submitted its reads can then silently recipient still waiting in its queue has submitted nothing, so it can later~~ compute on the next frame ~~without any error~~. A detector would ~~then~~ report objects seen in frame $k+1$ under the timestamp of frame k ; ~~and, at 20 Hz—Hz,~~ a vehicle driving at 20 m/s moves a full meter between the two. *Full retention* holds the slot until every result is ready, as reference-counted zero-copy transports do when recipients keep their references until done [5], [6]. With ~~a one-slot pool, a new frame is dropped while that slot is held one slot, frames that arrive during the hold are dropped.~~ In our 20 Hz trace, ~~the hold barely exceeds the frame period,~~ so full retention drops every other frame and leaves the GPU idle ~~between accepted frames in between~~ (Fig. 2).

Key insight: ~~A Our key observation is that a~~ shared tensor, ~~which we call~~ the *source*, has two lifetimes (Fig. 1b). Its *storage lifetime* ends when the last operation that reads it completes. Its *provenance lifetime* lasts until every result is delivered, because those results ~~must~~ still name the frame they came from, much as an essay keeps citing a library book after the book has gone back to the shelf. The two can differ widely. In the TransFusion detection head that reads our BEV map, only the first of 287 operations reads ~~it; the remaining operations work the source; the rest compute~~ on intermediate results in private storage and ~~can continue after the shared input are unaffected when the source~~ is overwritten. ~~Each A~~ recipient therefore needs its *borrow*, ~~its~~ permission to read the slot, only until its last source read completes.

~~Seen this way, full Full~~ retention, copying, and early reuse

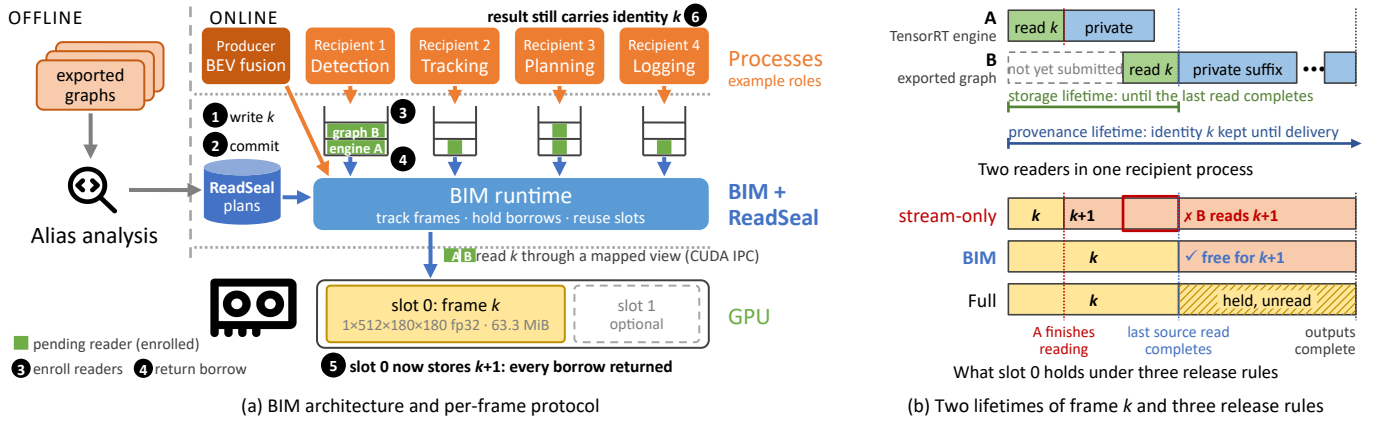


Fig. 1. **BIM at a glance.** (a) Offline, ReadSeal analyzes each exported model graph into a plan. Online, the producer writes frame k into a slot and commits it (1, 2); each recipient enrolls its readers before they submit GPU work (3) and returns its borrow once when their source reads complete (4); once every borrow is returned, the producer may overwrite the slot after every borrow is returned (5), while results still carry identity k (6). (b) Time runs left to right; TensorRT engine A and exported graph B are the two readers of the last recipient still holding a borrow; time runs left to right. Stream-only release lets $k+1$ overwrite the slot before the graph B reads it; full retention holds the slot until the outputs complete; and BIM frees reuses it at after the last source read.

are three choices of where a differ only in where the borrow ends: when the recipient's outputs are complete, when its a private copy is complete, or when its at the last source read completes. All three must protect a recipient from the moment its frame is published, before it starts reading.

Ending the borrow at the last read is safe only if three questions have answers can be answered. (1) Which operation reads last? Views, tensors that share the input source's storage, extend its reads, so the read boundary can come long after the source itself last appears (Fig. 3). (2) Who may still read? A reader that has not yet submitted GPU work leaves no event to wait on. (3) Is the analyzed program the one that runs? A boundary derived for one model version can be wrong for the next. Compilers answer the first question for a single program [9], [10]; here the producer sees neither the recipients' programs nor the reads they have yet to submit.

We present BEV-in-Memory (BIM), a cross-process borrowing contract motivated by BEV sharing, and ReadSeal, the mechanism that answers these questions. BufferQueue and NvSciStream already let each consumer return a shared buffer on its own [11], [12], but they leave the application to decide where it should do so and how to cover components that have not started reading which decides where each recipient's borrow ends. The producer knows only which processes receive a frame, so BIM answers question (2) across processes; it registers every recipient's borrow when the frame is published, before any recipient starts (Section III). Each process knows which of its components will read the frame recipient knows its own components, so ReadSeal enrolls them before they submit answers the rest inside it: it enrolls all readers before any of them submits GPU work, derives each one's read reader's boundary by alias analysis of its model graph, and binds the plan to the code, model state, and input layout that execute (Section IV). BufferQueue and NvSciStream already let each consumer return a shared buffer on its own [11], [12], but they leave the application to decide at which point in its

computation to return it and how to cover components that have not started reading.

Deriving the boundary matters because a A hand-placed one boundary can fail in two ways. It can go stale or be wrong from the start. One: one added late read moves our detection head's boundary from the first operation to the second-to-last (Fig. 3). While Or it can be wrong from the start: the first operation is right the right boundary for the head, but in a ResNet-18 tail [13] the residual connection reads the source again at operation 12. A version check catches the first failure but not the second. ReadSeal recomputes avoids both, because it derives the boundary for each model version and refuses to run a plan that no longer matches execution. Our contributions are:

- **The A two-lifetime view of a shared GPU tensor tensors** (Sections I–II). Separating storage from provenance shows when a buffer can be reused while results keep their frame identity, and it places full retention, copying, and early reuse on one axis.
- **Checked release at a derived last read** (Sections III–IV). BIM protects every recipient from publication onward, including readers that have not submitted work, and ReadSeal derives where each reader stops reading and rejects plans that no longer match execution.
- **An evaluation of the release choices where borrows should end** (Section V), comparing early reuse with early reuse against full retention, extra slots, and copying under load, and testing the derived boundaries under ordered overwrites and model updates.

With early source reads, one BIM slot delivers $1.31\text{--}1.50\times$ the timely results of one-slot full retention at 25–40 Hz and, at 30 Hz, matches two full-retention slots with a tuned request cap while using one fewer 64 MiB buffer (Section V-C); mean latency rises by 8–11 ms. Early release also changes how a pool should be sized: given a second slot and no cap, BIM admits more work than the GPU can finish in time, and its timely

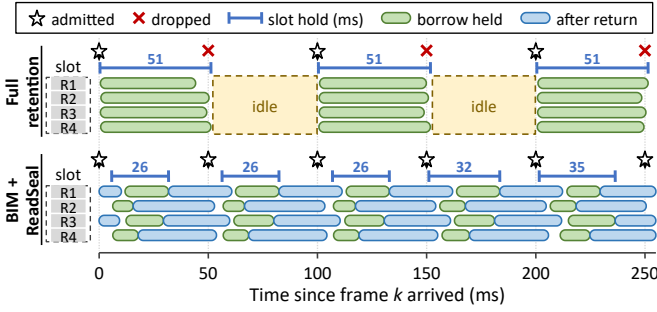


Fig. 2. **Full retention refuses frames while the GPU idles.** Measured one-slot trace at 20Hz (repetition 0); R1–R4 are the recipients, and labels give slot hold times in ms. Full’s 51 ms hold just exceeds the 50 ms frame period, so it refuses every other frame even though the recipients sit idle; full retention is not work-conserving. Under BIM, each recipient’s work after its borrow return overlaps the next frame.

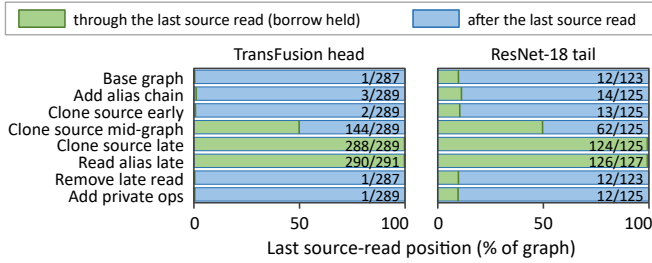


Fig. 3. **Model edits move the last source read.** Each bar is a graph snapshot; green marks the operations through the last source read, during which the recipient holds its borrow. Each snapshot that applies one edit to the base graph (Remove late read undoes Clone source late). Green marks the operations up to the last source read, during which the recipient holds its borrow; labels give the last source reading that operation’s index / total operations. Add alias chain inserts two storage-sharing views, which shift the operation indices but leave renumber operations without moving the runtime-release point unchanged.

results fall from 78.5 to 24.9 per second. Derived boundaries deliver as much as hand-placed ones and follow model changes automatically. When the last read comes late, a recipient reads the source late, early release gains nothing, and per-recipient copies are the better choice if memory allows.

II. BACKGROUND AND MOTIVATION

Each release rule below ends a recipient’s borrow at a different point; Table I compares them.

GPU background. A kernel is a function executed on the GPU. A process places kernels in a stream, which executes them in order, and continues without waiting. A read ends when its kernel completes. An event marks a position in a stream and completes after the preceding work. PyTorch’s record_stream lets the caching allocator reuse a buffer only after the work already enqueued on the recorded streams has completed [14]. CUDA inter-process communication (IPC) lets another process map the same device buffer and wait on shared events. But an event can only mark work that is already enqueued, so no process can wait on reads that another process has not yet submitted.

Releasing too early fails silently. The simplest case is a recipient still waiting in its queue. It has enqueued no GPU

work, so a rule that waits only for submitted work, such as record_stream or a read event, returns-releases the slot once the other recipients’ reads complete. The producer writes $k+1$, and the waiting, although a recipient still waiting in its queue has enqueued nothing. That recipient later computes a numerically valid result on it frame $k+1$ while its request still names k . No, and no crash or error signals the failure, so gives a downstream planner has no reason to distrust the result. The same gap can open inside a recipient that combines a TensorRT engine (NVIDIA’s inference runtime) with a framework graph. In our running example (Fig. 1b), the two readers are TensorRT engine A, which signals an input-consumed event after reading its input [15], and exported graph B. An adapter forwards a view to the graph without reading the source itself. Once the engine’s event completes, stream-only release returns-the-slot-makes the slot reusable although the graph has not submitted its reads; Section V-E tests this ordering. ROS 2’s CUDA buffer backend exposes the same choice. A subscriber’s read handle records a read event when it is destroyed, and the block is recycled once every handle is released [16]. Destroying the handle after the engine’s read gives stream-only release; holding it until the outputs complete gives full retention.

Holding the source through computation. Full retention is not work-conserving: its hold refuses frames while the GPU idles (Fig. 2). A second slot absorbs some of those arrivals but can admit more work than the GPU finishes before the deadline (Section V-C).

Copying the source. A private copy lets a recipient compute after the shared slot is reused. The copy itself reads the source, however, so the slot must remain-protected-while-a-recipient-is-queued-stay protected while the recipient waits in its queue and until its copy completes. Our clone-on-accept policy therefore uses BIM’s recipient registration and ends each borrow at copy completion.

Maintaining a read boundary. A view can carry a source read into later operations (Fig. 3), so a hand-placed boundary must be reviewed whenever the model changes. A stale boundary releases too early unless execution is guarded against the change.

III. BIM: FRAME IDENTITY AND BUFFER REUSE

BIM separates memory, frame identity, and permission to read. The producer has-owns a pool of N reusable GPU slots on one machine. It writes frame k into a slot and attaches an identity that every result from that frame carries. The, the same slot can later hold frame $k+1$.

A-recipient-process-A recipient holds one borrow until-all its-reader-components-per frame, covering all of its readers, such as engine A and graph B, have-finished-reading-the source, including readers still waiting to submit work. We keep three terms apart: a release rule decides when a recipient’s borrow ends, the recipient then returns its borrow by clearing its borrow bit, and the slot becomes reusable when its last bit clears.

TABLE I

RELEASE ~~OPTIONS~~ POLICIES FOR FOUR RECIPIENTS (K : REQUEST CAP).
 EXTRA MEMORY COUNTS SOURCE SLOTS AND INPUT COPIES BEYOND ONE
~~SLOT; K IS THE REQUEST CAP (SECTION III). $\dagger$ MANUAL USES AN~~
~~INDEPENDENT DIGEST-CHECKED RUNTIME; MANUAL-CHECKED USES~~
~~BIM'S RUNTIME AND CHECKS MANUAL-CHECKED (SECTION V-A).~~
 $\ddagger$ ~~MISSSES STREAM-ONLY RELEASE CANNOT WAIT FOR~~ READERS THAT
 HAVE NOT SUBMITTED GPU WORK.

Policy	Extra memory	Slot reusable after	Read boundary
Stream-only	—	submitted reads	stream-idle
Full retention	—	all outputs	—
Full, 2 slots, $K=8$	one slot 64 MiB	all outputs	—
Clone-on-accept	256 MiB	all copies	fixed no Manual
Manual-checked <u>Hand-placed</u> $\dagger$	—	all last reads	by hand
BIM + ReadSeal	—	all last reads	derived

Descriptors and views. Recipients receive small descriptors over a control plane such as ROS 2 and map the resident tensor, here through PyTorch's CUDA IPC support [17]. A descriptor carries ~~a the~~ frame identity (producer instance and sequence number), a storage locator, and ~~an interpretation how to read it;~~ element type, shape, strides, and a checked ~~view range. The view specifies which bytes a recipient may address and how to interpret them~~ range of addressable bytes. BIM rejects stale identities and incompatible interpretations.

Borrow bits and admission. A process-shared lock protects slot reservation, the frame sequence, and one borrow bit per recipient, set while that recipient may still read the slot. An arriving frame takes a slot whose bits are all clear; if none is free, the producer drops the frame instead of waiting. Once the frame is written, the producer *commits* it by setting every recipient's bit before sending any descriptor. ~~This ordering is what protects a~~ A recipient still waiting in its queue is therefore protected, because its bit ~~is was~~ set before it could possibly read. ~~Each recipient clears its bit when it returns its borrow, and clearing~~ Clearing the last bit makes the slot reusable without ~~freeing-deallocating~~ or unmapping it. If frame k is committed at time c_k and recipient j returns its borrow at time $t_{j,k}$, the slot is held for H_k . With one slot, a frame arriving at time a is admitted only if

$$a \geq c_k + H_k, \quad H_k = \max_j t_{j,k} - c_k. \quad (1)$$

Full retention sets $t_{j,k}$ to the time j 's outputs complete, and ReadSeal to the time j 's last source read completes (Section IV-B). ~~This difference in hold time also changes which new frames can enter, so ReadSeal's shorter hold admits more of the arriving frames.~~ An optional request cap K bounds ~~outstanding recipient requests, counted from admission to the recipient requests outstanding between admission and receipt at the output sink. A frame must pass both admission gates: a free slot and room under the cap; a frame is admitted only if a slot is free and the cap has room for it.~~

IV. READSEAL: CHECKED BORROWING

ReadSeal prepares a *plan* once per model version, ~~then and~~ checks it each time a recipient processes a frame (Algorithm 1).

Algorithm 1 BIM's commit-publication and ReadSeal's plan and invocation. $b_{s,j}$: borrow bit of recipient j on slot s ; U_j : readers of j ; T : binding tokens.

```

1: procedure PUBLISH(frame  $k$ ) ▷ producer (BIM)
2:   if no slot is free or cap  $K$  requests are outstanding has no room for  $k$ ,
   drop  $k$ 
3:   write  $k$  into a free slot  $s$ ; commit: set  $b_{s,j}$  for all  $j$ 
4:   send each recipient a descriptor of  $(k, s)$ 
5: procedure BUILDPLAN(graph  $G$ ) ▷ offline, per model version
6:   compute  $Q_r$  by (2) and  $\beta$  by (3) Wrong
7:   split  $G$  at  $\beta$ ; record readers  $U$  frame? tokens  $T$ 
8: procedure INVOKE(plan, frame  $k$ ) possible $\dagger$  ▷ recipient  $j$ 
9:   check  $T$ ; keep identity  $k$  for every no output
10:  enroll every reader in  $U$   $U_j$ , before any of them submits GPU work
11:  submit the stages in declared order, checking  $T$  at each entry
12:  on failed check: cancel unsubmitted reads, drain, publish nothing
13:  on each completed or cancelled read: if (4) holds, clear  $b_{s,j}$ 
14:  on completed outputs: publish them with identity  $k$ 

```

The plan records where the graph stops reading shared storage, which components may read it, and which program version was analyzed. One execution of ~~the a~~ plan is an *invocation*. ~~Its~~ its recipient returns the borrow when every planned source read has finished or was cancelled before starting.

A. Deriving the Read Boundary

The analysis runs on each graph, a fixed-shape, functional directed acyclic graph (DAG) of operations. ~~Of the two lifetimes, only storage needs analysis. Every~~ Only the storage lifetime needs analysis, because every result carries the frame identity attached at acceptance. Storage sharing propagates through views. A slice of the source is a view, so an operation on the slice still reads the source, whereas a convolution reads the source but writes fresh storage, so operations on its output do not. ~~In our running example, the adapter only forwards a view; the graph receiving that view reads the source.~~

Formally, let $S(v)$ be the set of shared roots whose storage a value v may alias, with $S(r) = \{r\}$ for the source r . For an operation o with inputs v_i and output v_o , let $\mathcal{A}_o$ index the inputs the output may alias and $\mathcal{R}_o$ the inputs o reads. Then

$$S(v_o) = \bigcup_{i \in \mathcal{A}_o} S(v_i), \quad Q_r = \{o : \exists i \in \mathcal{R}_o, r \in S(v_i)\}, \quad (2)$$

where Q_r is the set of operations that may read the source. With the operations numbered $o_1, \dots, o_n$ in execution order, the read boundary is the position of the last one in Q_r ,

$$\beta = \max\{i : o_i \in Q_r\}, \quad (3)$$

so the *source-reading prefix*, $o_1, \dots, o_\beta$, includes all source reads. ~~The, and the private suffix, $o_{\beta+1}, \dots, o_n$, computes the remaining results using in private intermediate storage. If no operation reads the source, Q_r is empty and we set $\beta = 0$.~~ Fig. 3 labels each graph snapshot with β/n ; the base head has $\beta = 1$ and $n = 287$.

Operator summaries cover 42 reviewed operator variants in ATen, PyTorch's operator library, plus fixed-index tuple selection. Each summary counts every operand as read and treats an output as sharing storage whenever the operator's declared

signature permits aliasing with an input. Plan construction rejects unsupported mutation, opaque callbacks, and source views that escape the graph. The implementation compiles the prefix and suffix into separate executable stages. ~~A~~, and a CUDA event after the prefix ~~then~~ marks the end of its source reads.

B. Enrolling Readers Before Submission

~~Registration follows the two consumer levels in Section III.~~ Protection works at two levels. BIM protects each recipient process by setting its borrow bit at commit. When that process accepts a frame, ReadSeal registers, or *enrolls*, all ~~its reader components of its readers~~ before any of them submits GPU work. Each reader u marks its last source read with a completion event: A's input-consumed event or the event after B's prefix. Let U_j be the readers of recipient j . Then j returns its borrow for frame k at the first moment when

$$\forall u \in U_j : \text{done}_u \vee \text{cancelled}_u, \quad (4)$$

where ~~the predicate~~ done_u ~~holds once becomes true when~~ u 's completion event ~~has completed completes~~, and cancelled_u ~~once when~~ u ~~has been is~~ cancelled before submission. Every enrolled reader ~~participates takes part~~ in this condition. ~~When including one that has enqueued nothing: when~~ A's event completes, B's read is still pending, so the bit stays set ~~even though B has enqueued no GPU work. Returning the borrow clears j's bit~~. Once all recipients have returned their borrows, the producer can write the next frame while j 's private suffix ~~continues keeps~~ computing results labeled k .

C. Binding Plans to Execution

At construction, ReadSeal captures the functions to run and copies the model's parameters and buffers once into plan-owned storage, so later edits by the caller cannot change a plan in use. ~~The manual implementation does the same; the hand-placed baselines keep the same copy.~~

ReadSeal records *binding tokens* T ; identifiers and metadata for code and globals, model storage and version, input layout, and compiled stages. At acceptance and ~~at~~ each stage entry it compares them with the values $\hat{T}$ for the execution about to run ~~, $\hat{T}$, and~~ proceeds only if $\hat{T} = T$. A failed check cancels unsubmitted reads, drains started work, and publishes nothing. The application supplies the exported graph, each native engine's last-input-read event, and the component and output dependencies. ~~New; new~~ operators require reviewed summaries.

D. Correctness Conditions

The analysis must include every operation that can read the source. It assumes that operator summaries conservatively describe reads and storage sharing, kernels obey those summaries, and the graph executes in the analyzed order.

Proposition 1. *Under these assumptions, Q_r in Eq. (2) contains every operation that can read the shared source. No operation after the derived boundary β reads that source.*

Proof sketch. Initially $S(r) = \{r\}$. At each operation, the union in Eq. (2) preserves the shared roots of every input that its output may alias. Induction over graph order therefore includes every source alias in the corresponding $S(v)$. The read rule places every operation consuming such storage in Q_r ; by Eq. (3), none follows $\alpha\beta$. $\square$

~~For the runtime, The runtime further requires that~~ each reader's completion event ~~must occur after follow~~ its source reads, ~~mappings must remain that mappings stay~~ valid, and ~~identities must remain that identities stay~~ unique while referenced. ~~Under these conditions, enrolling readers before submission~~ (Algorithm 1 ~~registers every reader before submission, checks the plan at execution, and returns~~), ~~returning~~ a borrow only when Eq. (4) holds. ~~BIM permits overwrite after all recipient bits clear. Together, these steps, and overwriting only after all bits clear~~ preserve the source until every reader finishes, while results keep the identity captured at acceptance.

V. EVALUATION

~~Section V-B measures what ending borrows at the last read gains under load and costs in latency; Section V-C compares one early-released slot with two fully retained ones; Section V-D adds jitter, a late reader, and copying; Section V-E tests safety under reordering and model changes; and Section V-F measures what the checks cost.~~

A. Setup

Scope and platform. ~~Platform.~~ We evaluate cross-process reuse ~~of saved BEV features~~ on one NVIDIA A100 PCIe 40 GB GPU. The producer, four recipients, and ~~output sink an output sink are separate processes that~~ communicate through multiprocessing queues and share storage and events through CUDA IPC. ~~Saved features To~~ isolate reuse and admission from feature extraction; ~~the setup does not run ROS 2, the BEV front end, or an embedded driving stack, the producer replays saved BEV features.~~ The runtime uses PyTorch 2.1.2, CUDA 12.1, and TensorRT 10.3. Without NVIDIA's Multi-Process Service (MPS), recipient kernels take turns on the GPU. Records, validators, ~~campaign details~~, and analysis scripts are public.¹

Workload. ~~The exported graph (B) Graph B~~ is BEVFusion's functional TransFusion head [3], [18], ~~exported~~ with 287 operations. The producer publishes precomputed 63.3 MiB feature maps from three fixed nuScenes-mini frames [19]. ~~The native engine (A) Engine A~~ runs the head's shared convolution, ReLU, and global average pooling in TensorRT; ~~and~~ an adapter forwards a view to the graph. This controlled composition gives each recipient two readers of the same source, each with its own completion event. Four recipients run identical compositions in the rate and slot sweeps; Section V-D varies their components. A pretrained ResNet-18 tail [13], ~~made of~~ the residual stages and classifier with saved stem activations as the ~~shared~~ source, serves in the correctness, model-change, and cost tests.

¹<https://github.com/assasin831/readseal-artifact/tree/ispa2026-r17>.

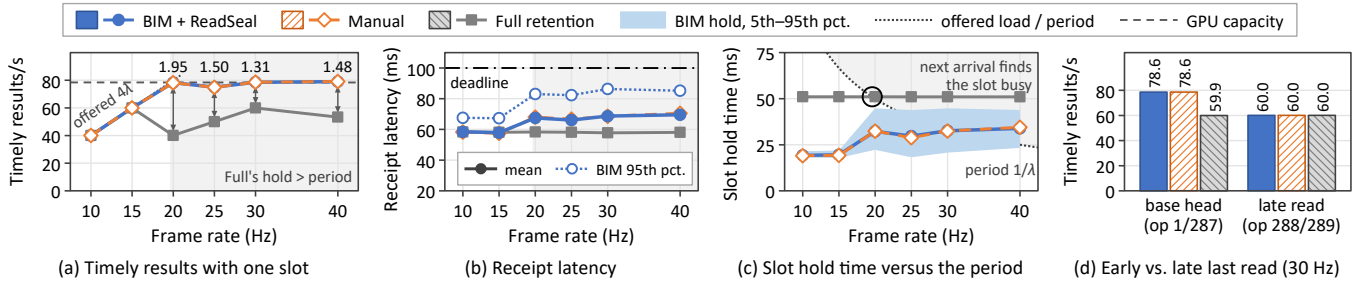


Fig. 4. ~~Early reuse raises one-slot delivery under load, with a latency cost.~~ Early reuse raises one-slot delivery under load, at a latency cost. One slot, no pipeline-wide cap K ; six runs per condition and policy. ~~Each input; each frame offers four recipient results; results/s counts individual results.~~ (a) Labels give BIM/Full ratios. (b) Mean receipt latency ~~and BIM's mean per-run 95th percentile is computed per run and averaged.~~ (c) Mean hold and BIM's 5th-95th percentile; shading marks where Full's hold exceeds the frame period. (d) Moving the last read to the graph's end removes the gain.

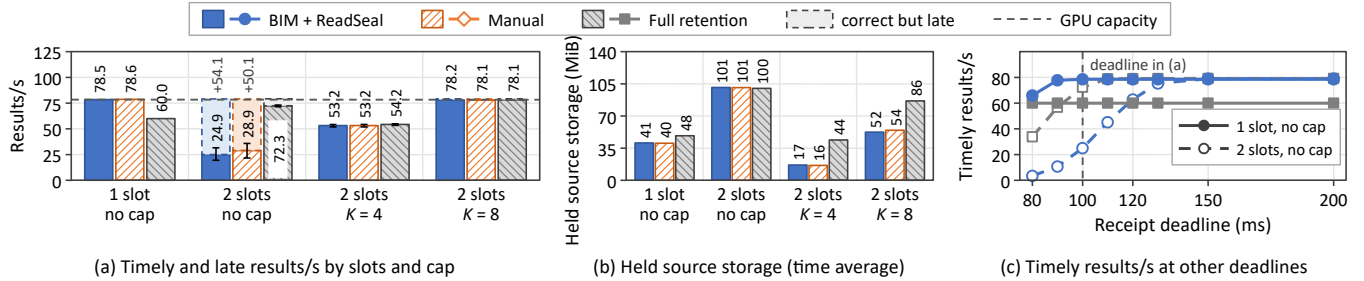


Fig. 5. One BIM slot matches full retention with two slots and a request cap (30 Hz; six runs per condition and policy). K counts outstanding recipient requests; ~~eight requests are two full groups of four.~~ (a) Solid: timely results; dashed: correct but late; whiskers: 95% intervals. (b) Time-averaged source storage held by unreturned borrows; ~~not allocated memory.~~ Device peaks are 5.078 MiB with one slot and 5.142 MiB with two for every policy. (c) BIM and Full ~~The same runs~~ re-scored at ~~different other~~ deadlines using the same runs.

Policies. All policies share the pool, workloads, queues, and admission settings. BIM uses ReadSeal, and Full retains the source until its outputs complete, and Clone-on-accept gives each recipient a private input copy when it accepts a request. Two hand-placed baselines ~~come from different campaigns answer different questions.~~ Manual ~~used in the earlier rate and slot campaigns, is a separately is an independently implemented runtime that registers enrolls both readers and checks code and model digests, so it tests early reuse independently of our runtime apart from our code (Sections V-B and V-C).~~ Manual-checked ~~used in the later jitter and mixed-recipient campaign, shares runs on BIM's runtime and binding checks and differs only in where the boundary comes from, which isolates automatic placement.~~ Clone-on-accept gives each recipient a private input copy when it accepts a request so it isolates the effect of deriving the boundary (Section V-D).

Metrics. The producer schedules frames at rate λ , in Hz. Each frame should yield four results, one output bundle per recipient, so the offered load is 4λ results/s. A result is *timely* if it passes source-identity and reference-output checks and reaches the output sink, after device-to-host transfer and IPC, within 100 ms of the frame's planned arrival. ~~The 100 ms budget is a common benchmark threshold rather than a vehicle requirement, so we also re-score; Fig. 5c re-scores the slot runs at 80-200 ms.~~ Receipt latency runs from planned arrival to sink receipt; and slot hold time runs from commit to

the last borrow return. ~~The complete-frame timely fraction (Section V-D) counts a frame only if all four intended results are also reports the complete-frame timely fraction: the share of scheduled frames, refused and partially admitted ones included, whose four results are all correct and timely, and its denominator includes refused and partially-admitted frames.~~

Queue settings Queues and statistics. ~~The In the rate and slot sweeps permit, each recipient permits at most two pending requests and four in flight per recipient. Optional. When set, the cap K limits outstanding recipient requests across the pipeline; $K=8$ accommodates two full groups of four requests. No pipeline-wide cap leaves the per-recipient queue limits in place. Because the. The producer drops frames rather than blocking, so every policy sees the same offered load.~~

Statistics. ~~Rate and slot~~ These runs have 2 s of warm-up, 12 s of measured arrivals, and a complete drain. ~~Condition with condition and method order are randomized, with randomized and six paired repetitions per condition; brackets give pointwise 95% intervals from 20,000 paired whole-run bootstrap draws. The two campaigns in-of Section V-D use queues of eight, 10 s of warm-up, and 30 s of measurement, with six paired runs, and pointwise 95% t intervals. Campaigns are analyzed separately. Latency summaries describe individual recipient results in the rate sweep (mean and per-run 95th percentile) and complete correct frames in the mixed-recipient study (per-run 99th percentile); each campaign is analyzed on its own.~~

B. Early Reuse Raises Delivery Under Load

Early reuse pays off once the frame period drops below full retention's hold time (Fig. 4a). Both policies keep up at 10 and 15 Hz. At 20 Hz, where Full accepts only every other frame, the ratio of timely results is BIM delivers $1.95 \times$ Full's timely results, and at 25–40 Hz it is delivers 1.31 – $1.50 \times$ as many, running near the GPU's capacity. BIM stays within 0.2 results/s of Manual at every rate.

Slot hold time. The gain comes from a shorter hold. Full holds the slot for about 51 ms, nearly the entire computation of the four recipients, whereas BIM returns it after their source reads and cuts the hold to, in 30–34 ms (Fig. 4c). The hold is not shorter still because, although the recipients take turns on the GPU: each head reads the source in its first operation, but the last recipient must wait for its turn on the shared GPU before that operation runs. The shorter hold lets the next frame enter while private computation continues, which fills the idle gaps of Fig. 2. The dip at 25 Hz comes from periodic admission, since after a refused frame the producer waits for its next scheduled arrival: the 40 ms period is shorter than BIM's longer holds (95th percentile 44 ms), so some frames find the slot busy, and each refusal leaves it waiting a full period for the next arrival, which comes sooner at 30 and 40 Hz.

Receipt latency. The additional The extra admitted work queues behind other recipients on the GPU, which costs latency. At 20–40 Hz, BIM's mean latency is 8–11 ms higher than Full's and its per-run 95th percentile 15–19 ms higher (Fig. 4b). The delivery lead at At 30 Hz holds across 100–200 ms deadlines BIM's delivery lead holds for deadlines from 100 to 200 ms and narrows at 80 ms, where more of this queued work arrives late (Fig. 5c).

Late source reads. Moving the last source read The gain also depends on where the last read falls. Moving it to operation 288 of 289 leaves almost no private computation to overlap (Fig. 3's late clone) leaves almost no private computation to overlap. BIM's hold then reaches Full's, and all three policies deliver 60.0 results/s at 30 Hz (Fig. 4d). The gain comes from overlapping the private suffix with the next frame.

C. One Slot Matches Two, and Two Need a Cap

A second slot is the usual remedy for full retention's refusals (Fig. 5a). At 30 Hz, two-slot Full with $K=8$ delivers 78.1 results/s, matching one-slot BIM's 78.5, so BIM reaches the same delivery with one fewer source buffer. In this pipeline that buffer is 64 MiB of a device peak of roughly 5 GiB source buffer. The second slot also makes the cap decisive: $K=4$ holds every policy to 53–54 results/s, and no cap lets work miss without a cap work misses the deadline.

Queueing with two slots. With one slot, the hold time in Eq. (1) paces BIM's admissions. A second slot removes that pacing and lets requests queue before the GPU can start them. Without a pipeline-wide cap, two-slot BIM delivers only 24.9 timely results/s against Full's 72.3, and another 54.1 results/s arrive correct but late. Full's longer hold still throttles its

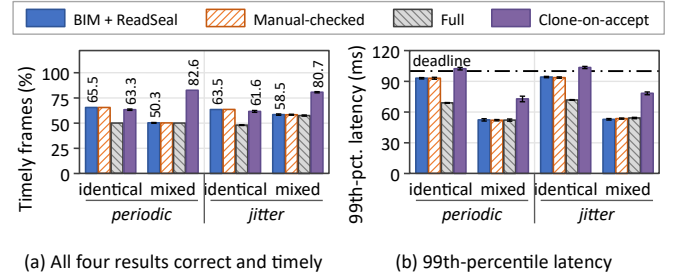


Fig. 6. Complete-frame delivery and tail latency with jitter and mixed recipients (30 Hz, one slot, no pipeline-wide cap; 6 six runs per bar; whiskers: 95% t intervals). (a) Share of the 900 scheduled frames per run whose four results are all correct and timely; (b) per-run 99th-percentile latency of frames whose four results are all correct. Identical: four copies of the composition; mixed: four component profiles, one reading late.

arrivals, so it delivers 72.3. Manual shows the same drop and a looser deadline recovers as BIM, and looser deadlines recover the delayed results (Fig. 5c).

Request cap. At $K=8$, all three policies reach similar delivery across the two-slot rate sweep. BIM still returns borrows sooner, so the time-averaged source storage held by outstanding borrows falls from 86.4 MiB under Full to 52.3 averages 52.3 MiB against Full's 86.4 MiB (Fig. 5b). The metric counts locked storage rather than allocation; returning a borrow makes a slot reusable without freeing deallocating it, so the device peak is unchanged the same under every policy: 5,078 MiB with one slot and 5,142 MiB with two.

D. Jitter, Mixed Recipients, and Copying

This study asks whether every intended recipient finishes on time, using the complete-frame metric (Fig. 6). At 30 Hz with one slot and no pipeline-wide cap, arrivals are periodic or displaced by up to 0.8 of a period (jitter). Recipients are either identical copies of the composition or mixed: the native engine alone, the exported head alone, their composition, and a composition that reads the source late.

Arrival pattern and recipient mix. With identical recipients, BIM completes 65.5% of frames on time under periodic arrivals and 63.5% with jitter, about 15.5 percentage points above Full in both cases. With mixed recipients, the late reader holds the slot longer for all four, and BIM stays less than one point above Full (Fig. 6a). Jitter raises mixed-recipient delivery from 50.3% to 58.5% for BIM and from 50.0% to 57.5% for Full. This is consistent with irregular arrival gaps allowing gaps letting more frames past a hold that otherwise refuses every second periodic arrival. Across this study, including 12 all-late runs, Manual-checked stays within 0.13 percentage points of BIM: with the runtime and checks held fixed, deriving the boundary costs nothing in delivery.

Input copying. Copying helps where early reuse cannot. Clone-on-accept releases the shared input after copying, even when computation will read returns its borrow when the copy completes, even if computation reads the private copy much later. With mixed recipients it completes 22–32 percentage points more frames on time than BIM, at a 256 MiB higher

sampled device peak and 20–25 ms more 99th-percentile latency. ~~With~~; with identical early-reading recipients it completes about two points fewer.

A separate 168-run study counts individual ~~recipient~~ results. With early readers, BIM delivers 0.4–0.9 results/s more than cloning at 20, 30, and 40 Hz; while cloning leads by 2.75 [2.47, 3.03] at 25 Hz, where BIM shows its admission dip. When all readers read late at 30 Hz, cloning delivers 76.4 results/s against BIM's 60.0, a paired gain of 16.39 [16.06, 16.72], but it raises the mean per-run 99th-percentile latency from 68.7 to 101.1 ms and ~~worsens freshness. Its the~~ sampled device peak ~~is from~~ 5,334 MiB, against BIM's ~~078 to~~ 5,078 ~~334~~ MiB.

~~Hand-placed comparison.~~ Across this study, including its 12 all-late runs, Manual-checked stays within 0.13 percentage points of BIM. ~~With the runtime and checks held fixed, deriving the boundary automatically costs nothing in delivery.~~

E. Safety Across Reordering and Model Changes

~~Future readers.~~ We first reorder reads on purpose. In six controlled ~~ordered~~ tests, engine A's input-consumed event completes before graph B submits its reads. Under stream-only release, the graph's outputs matched the replacement frame in every test, because recording stream use cannot cover reads that have not been submitted [14]. ~~Under~~; under BIM and Manual, they matched the admitted frame. Under load, all each of the 109,592 results delivered by BIM, Manual, and Full in the 144 rate and late-read runs carried ~~their-its~~ admitted identity and matched the reference outputs.

~~Boundaries under model change.~~ Regenerated boundaries ~~kept every output exact~~ We then change the models. Across eight graph snapshots per model, three inputs, and two methods, regenerated boundaries kept all 96 ordered-overwrite cases (~~540 output tensors~~) ~~matched exact against~~ unsplit references, and binding validation rejected ~~all each of~~ 22 attempted substitutions of plan, code, globals, model state, or input before the affected stage ran.

~~Boundary maintenance.~~ We Finally, we replay 16 controlled transitions: 14 graph edits, one batch-normalization folding, and a public ResNet-18 to ResNet-50 substitution, whose binding check runs on the CPU. The runtime release point moves in 13 of them; folding keeps the head's first source read, and the two alias-chain edits only renumber operations (Fig. 3). Binding checks reject ~~all 16 every~~ outdated hand-placed ~~plans (the model substitution is checked on the CPU) plan,~~ so a manual deployment ~~waits stops~~ until someone refreshes its plan. ReadSeal rebuilds the updated plans in 0.08–0.21 s on the CPU ~~without~~, with no source-reader or boundary annotations. ~~The In the mixed-recipient tests show the same pattern. After campaign, adding a late read was added, likewise made~~ Manual-checked's old boundary ~~failed fail~~ its check until the boundary was moved, ~~and after which~~ all 24 update cases ~~then~~ gave exact outputs.

F. Execution Cost and Coverage

An isolated-process test times recipient acceptance to output dispatch (8 process repetitions of 50 randomized invocations

per method; three inputs; base and late variants of both models). ReadSeal's checks add 0.8–0.9 ms per invocation, mostly at acceptance and stage submission. ~~That is~~; 8% over Manual for the base head (11.8 ms against 10.9 ms) and 22–23% for the lighter ResNet tail. In the pipelines above ~~the this~~ cost overlaps GPU work, ~~and~~; even at 10 and 15 ~~Hz Hz~~, BIM's mean receipt latency is within 0.4 ms of Manual's. A static survey of 14 torchvision tails accepts nine, with boundaries in their first 1–12 operations, and rejects five for unsupported operators.

VI. DISCUSSION

~~Why derive the boundary?~~ Hand-placed boundaries delivered as much as derived ones in every campaign, so ReadSeal's value is correctness as models change. Binding checks reject a plan built for other code, weights, or inputs, but they cannot tell whether a current boundary was right to begin with, such as one placed after the ResNet tail's first operation or one that misses a late read through a view (Fig. 3). ReadSeal finds both in the graph. It still relies on reviewed operator summaries, native input-consumed events, and declared component dependencies; missing summaries caused all five survey rejections.

~~Choosing a release policy.~~ Where each recipient's last source read falls, and how much memory is free, decide where its borrow should end. With early reads, one BIM slot matched two full-retention slots with a tuned cap and delivered about as much as per-recipient copies without their 256 MiB. With a late read, early reuse gains nothing (Fig. 4d), and ~~in the mixed-recipient study~~ copying completed 22–32 points more mixed-recipient frames on time than BIM, at 20–25 ms more tail latency. ~~With memory to spare and a tunable cap~~ Where memory is plentiful, extra slots or copies are simpler; early reuse fits ~~the setting that motivated this work, where memory is systems whose memory capacity and bandwidth are~~ too scarce for per-recipient copies. ~~Early release also changes how a pool should be sized: with one slot and no cap, the hold time~~ Pool sizing changes too: one slot paced BIM's admissions ~~so that the GPU finished every admitted result on time by itself,~~ whereas two slots needed a request cap.

Portability. BIM needs cross-process storage mapping and completion events that cover each reader's source access. ROS 2's CUDA buffer read handles [16] and NvSciStream's per-consumer packet release [12] are natural integration points. ~~A~~; a port would create each ~~consumer~~ recipient's hold before the frame is published, release it on ReadSeal's completion event, and validate the platform's ordering guarantees.

Deployment Limitations. Our measurements come from one A100 without MPS, where recipients take turns on the GPU, and from replayed features rather than a ROS 2 stack with its BEV front end. On this 40 GB card, the 64 MiB slot that BIM saves is small beside a 5 GiB device peak; the saving matters where the GPU shares DRAM with the rest of the system, which we have not measured. The six ordered tests show that stream-only release can serve the wrong frame, not how often it does so under load. The prototype runs declared

topologies with single-stream stages, ~~and~~; a topology change requires a new plan. A ~~crashed-recipient-leaves-its-borrow-outstanding-and-blocks-slot-reuse~~, so recovery must confirm ~~that the recipient's slow recipient holds the slot for everyone, as the late reader did in Section V-D, and a crashed one blocks reuse until recovery confirms that its GPU work has stopped~~ ~~before its borrow is removed~~.

VII. RELATED WORK

Zero-copy robotics middleware. iceoryx and Agnocast give ROS 2 subscribers direct access to publisher-owned shared memory [5], [6]. For GPU data, NITROS keeps its zero-copy path within one process [7], and ROS 2's `rosidl::Buffer` moves tensor bytes between processes through CUDA IPC [8]. These systems decide how a buffer is shared but leave open when it may be reused while a subscriber has yet to read it.

Release fences. Android's `BufferQueue` lets a consumer release a pooled buffer with a fence while still reading it [11], and `NvSciStream` returns a multicast packet to its pool once every consumer has released it [12]. BIM's borrow bits follow ~~this the same~~ per-consumer protocol. ~~ReadSeal adds the fence's position, and ReadSeal adds where the fence goes~~, deriving from each ~~consumer~~ recipient's model where its reads end.

Asynchronous buffer protocols. Simpson's four-slot mechanism separates concurrent reads and writes across buffers, and Kopetz and Reisinger's NBW protocol detects overlapping writes and also provides a multibuffer variant [20]–[22]. Cyclical asynchronous buffers let readers acquire the latest complete value without blocking the writer [23]. BIM instead preserves the particular frame already assigned to each recipient, including while its GPU reads are unsubmitted, ~~and ReadSeal supplies the model-derived point at which that borrow ends~~.

Last use and safe reclamation. Compilers and runtimes reuse a buffer after its last use, as in TVM's static memory planning [9], and Rust's non-lexical lifetimes end a borrow there [10]; both see every use in one program, in program order. Task runtimes such as Legion and StarPU know future accesses because tasks declare their data when submitted [24], [25]. BIM's recipients are separate processes that declare nothing to the producer. Safe memory reclamation, such as read-copy-update (RCU) grace periods [26], reuses memory once no reader can hold it, but protects only readers that have announced themselves. In RCU terms, BIM's commit opens every recipient's read-side section on its behalf, and ReadSeal closes it at the derived last read. ReadSeal's bindings, like PyTorch 2's guards [27], reject execution whose assumptions changed.

VIII. CONCLUSION

When several processes read one GPU buffer, the producer can overwrite it as soon as the last read completes, often long before the last result is ready, provided it knows which readers are still to come and where each one stops reading. BIM registers every recipient when a frame is published, and ReadSeal derives each reader's last source read from its model graph and refuses to run a plan that no longer matches execution.

Where a borrow should end depends on where that last read falls and on how much memory is free. In our A100 pipeline, where the detection head reads the shared map only in its first operation, ~~one slot released this way delivered ending borrows there let one slot deliver~~ $1.31\text{--}1.50\times$ the timely results of one-slot full retention at 25–40 Hz and, at 30 Hz, ~~matched as many as~~ two full-retention slots with a tuned request cap, ~~at for~~ 8–11 ms more mean latency. When a read falls late, the gain disappears, and per-recipient copies win if memory allows. Derived boundaries delivered as much as hand-placed ones and ~~followed~~, ~~unlike them, were rebuilt automatically for every model edit we replayed.~~ ~~When the last read falls late, per-recipient copies are the better choice if memory allows.~~ Early reads may be A small static survey suggests that early reads are common: all nine torchvision tails that ReadSeal accepted stop reading their input within 12 operations.

Nothing in the contract is specific to BEV maps; it applies wherever several processes consume one large GPU tensor. Next, we will integrate BIM with ROS 2's CUDA buffer backend and `NvSciStream`, measure ~~the BIM and its~~ checks on embedded ~~CPU cores~~ platforms whose GPU shares DRAM with the CPU, support recipients that run concurrently, and ~~copy selectively for late readers use each reader's derived boundary to decide, recipient by recipient, whether to borrow or copy.~~

REFERENCES

- [1] Autoware Foundation, "Autoware: Open-source software for autonomous driving," <https://autoware.org/>, 2024.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, p. eabm6074, 2022.
- [3] Z. Liu *et al.*, "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 2774–2781.
- [4] NVIDIA, "CUDA for Tegra application note (CUDA 13.0)," <https://docs.nvidia.com/cuda/archive/13.0.0/cuda-for-tegra-appnote/index.html>, 2025.
- [5] Eclipse Foundation, "Eclipse iceoryx: True zero-copy inter-process communication," <https://github.com/eclipse-iceoryx/iceoryx>.
- [6] T. Ishikawa-Aso and S. Kato, "ROS 2 Agnocast: Supporting unsized message types for true zero-copy publish/subscribe IPC," in *Proc. 28th Int. Symp. Real-Time Distrib. Comput. (ISORC)*, 2025, pp. 1–10.
- [7] NVIDIA, "Isaac ROS NITROS: NVIDIA Isaac transport for ROS," https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_nitros/isaac_ros_nitros/index.html, 2026.
- [8] Open Robotics, "About `rosidl::Buffer` backends (ROS 2 Lyrical documentation)," <https://docs.ros.org/en/lyrical/Concepts/Intermediate/About-Buffer-Backends.html>, 2026.
- [9] T. Chen *et al.*, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2018, pp. 578–594.
- [10] N. Matsakis, "RFC 2094: Non-lexical lifetimes," <https://github.com/rust-lang/rfcs/blob/master/text/2094-nll.md>, 2017.
- [11] Android Open Source Project, "BufferQueue and Gralloc," <https://source.android.com/docs/core/graphics/arch-bq-gralloc>.
- [12] NVIDIA, "NvSciStream APIs (NVIDIA DRIVE OS 6.0 Linux SDK API reference)," https://developer.nvidia.com/docs/drive/drive-os/6.0.8/public/drive-os-linux-sdk/api_reference/group_nvsci_stream_apis.html.
- [13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [14] PyTorch Contributors, "`torch.Tensor.record_stream` (PyTorch 2.1 documentation)," https://pytorch.org/docs/2.1/generated/torch.Tensor.record_stream.html, 2023.

- [15] NVIDIA, “TensorRT 10.3: Execution context and input-consumed event contracts,” <https://github.com/NVIDIA/TensorRT/blob/release/10.3/include/NvInferRuntime.h>, 2024.
- [16] Open Robotics, “`cuda_buffer_backend` (ROS 2 `rosidl_buffer_backends`),” https://github.com/ros2/rosidl_buffer_backends, 2026.
- [17] PyTorch Contributors, “Multiprocessing package: Sharing CUDA tensors (PyTorch documentation),” <https://docs.pytorch.org/docs/stable/multiprocessing.html>.
- [18] X. Bai *et al.*, “TransFusion: Robust LiDAR-camera fusion for 3D object detection with transformers,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 1090–1099.
- [19] H. Caesar *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11 621–11 631.
- [20] H. R. Simpson, “Four-slot fully asynchronous communication mechanism,” *IEE Proceedings E (Computers and Digital Techniques)*, vol. 137, no. 1, pp. 17–30, 1990.
- [21] H. Kopetz and J. Reisinger, “The non-blocking write protocol NBW: A solution to a real-time synchronization problem,” in *Proc. IEEE Real-Time Systems Symposium*, 1993, pp. 131–137.
- [22] J. Rushby, “Model checking Simpson’s four-slot fully asynchronous communication mechanism,” Computer Science Laboratory, SRI International, Tech. Rep., Jul. 2002, <https://www.csl.sri.com/~rushby/papers/4slot.pdf>.
- [23] G. Buttazzo, “Why real-time computing?” *Automazione e Strumentazione*, pp. 82–88, Feb. 2007, <https://retis.santannapisa.it/~giorgio/paps/2007/automazione07.pdf>.
- [24] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken, “Legion: Expressing locality and independence with logical regions,” in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, 2012, pp. 1–11.
- [25] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, “StarPU: A unified platform for task scheduling on heterogeneous multicore architectures,” *Concurrency Comput.: Pract. Exper.*, vol. 23, no. 2, pp. 187–198, 2011.
- [26] P. E. McKenney and J. D. Slingwine, “Read-copy update: Using execution history to solve concurrency problems,” in *Proc. Parallel Distrib. Comput. Syst. (PDCS)*, Oct. 1998, pp. 509–518.
- [27] J. Ansel *et al.*, “PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation,” in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, vol. 2, 2024, pp. 929–947.